main

Section 1. Data Collection

The XLK stock price we are looking at is accounted by S&P Dow Jones Indices and State Street Global Advisory. S&P Dow Jones Indices is in charge of creating the company benchmark on entering the technology company stock pool; while the State Street Global Advisors selected particular stocks into the combination of the XLK stock. The data we are using is collected from the investing.com website, a financial data platform that provides public real-time market data directly from stock exchanges such as the NYSE and NASDAQ. The primary purpose of collecting the data is to provide a transparent record of the performance of the Technology Selected Sector Index. The subjects of this data series are the large-cap U.S. technology companies such as Microsoft, NVIDIA, and Apple whose market valuations and returns constitute the movement of the XLK fund.

The variables we are using includes:

- Date: The specific trading day of the observed data
- Price (Close): The final price at which the ETF traded when the stock exchange closed (typically 4:00 PM EST).
- Open: The price of the very first transaction when the market opened (9:30 AM EST).
- High & Low: The maximum and minimum prices reached during the trading session.
- Vol.: The total number of shares traded during the day (e.g., 15.90M means 15.9 million shares).
- Change: The percentage difference between the current day's closing price and the previous day's closing price.

The data we are using is collected from 1/28/2019 to 3/28/2026, which incorporates the AI outbreak time. One of the limitations of this dataset is that there's a possibility for State Street to rebalance the stock composition over time, thus the data might not be consistent to offer a robustic overtime analysis.

Section 2. Variable Creation and Data Cleaning

Original dataset contains date as a character which may cause problems when using it as our time predictor, so we converted to the Date type in R. We observed outliers that are doubled the price of the remaining data points. Looking at the data, we found that for most weekends, prices are carried over from the Friday of the same week. These outliers correspond to weekend days, as indicated by their dates, and their volumes are recorded as empty strings. We decided to remove these outliers because they are not consistent with the general pattern, which they doubled the Friday price. Lastly, we split the dataset into two groups using November 30, 2022 (the release date of ChatGPT) as the cutoff. This is used for the comparison of the two time seires.

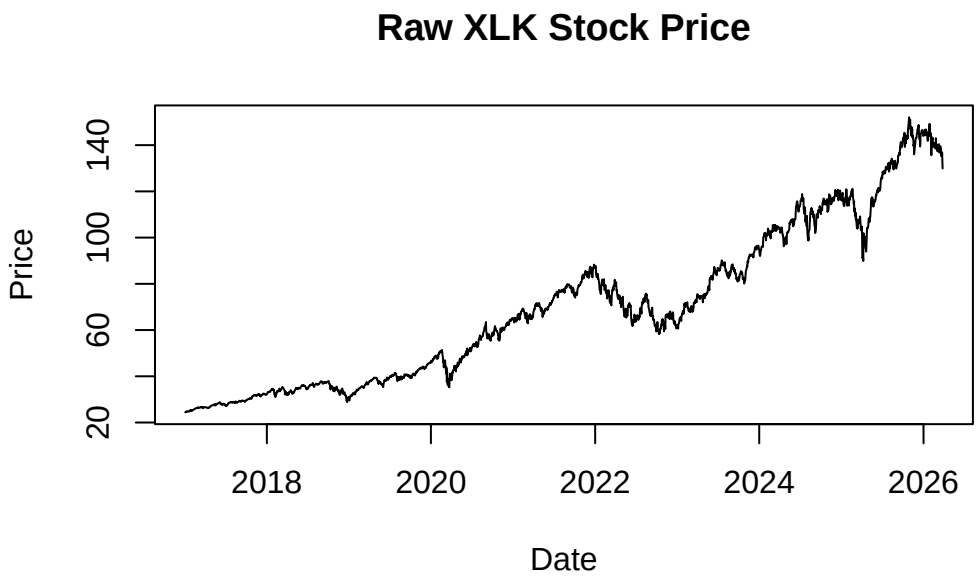
We will difference the data to stabilize variance, and remove the overall growth in the stock market as well as possible market cycles.

Section 3 Analytic Methods

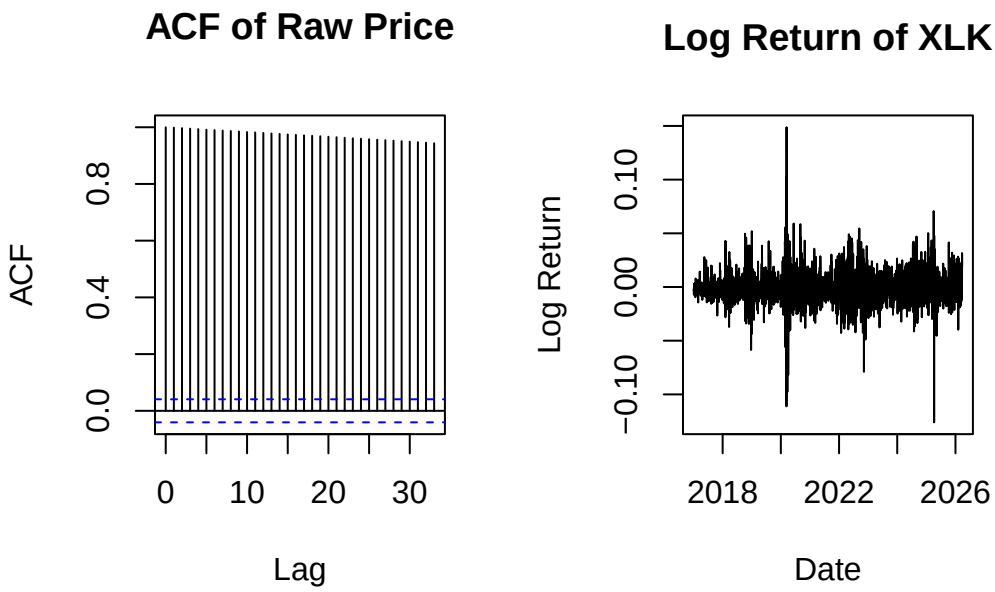
model assumption check

- check if data is white noise, or MA, or AR

```
# 1. Plot raw price series
plot(data$Date, data$Price, type = "l",
     xlab = "Date", ylab = "Price",
     main = "Raw XLK Stock Price")
```




```
# 2. ACF of raw price
par(mfrow = c(1, 2))
acf(data$Price, main = "ACF of Raw Price")

# 3. Transform to log return
data <- data |>
  mutate(
    log_price = log(Price),
    log_return = c(NA, diff(log_price))
  )

# remove NA in first row
log_return <- na.omit(data$log_return)

# 4. Plot log return
plot(data$Date[-1], log_return, type = "l",
     xlab = "Date", ylab = "Log Return",
     main = "Log Return of XLK")
```




```r
# 5. ACF of log return
par(mfrow = c(1, 2))
acf(log_return, main = "ACF of Log Return")


# 7. Check whether log return looks like white noise
Box.test(log_return, lag = 20, type = "Ljung-Box")
```

```
    Box-Ljung test

data:  log_return
X-squared = 198.51, df = 20, p-value < 2.2e-16
```

```r
# 8. Fit candidate models
AR1 <- arima(log_return, order = c(1, 0, 0))
MA1 <- arima(log_return, order = c(0, 0, 1))
ARMA11 <- arima(log_return, order = c(1, 0, 1))

# 9. Print model summaries
AR1
```

```
Call:
arima(x = log_return, order = c(1, 0, 0))

Coefficients:
          ar1  intercept
      -0.1426     -7e-04
s.e.   0.0206      3e-04

sigma^2 estimated as 0.0002436:  log likelihood = 6359.22,  aic = -12712.45
```

```r
MA1
```

```
Call:
arima(x = log_return, order = c(0, 0, 1))

Coefficients:
          ma1  intercept
      -0.1338     -7e-04
s.e.   0.0199      3e-04

sigma^2 estimated as 0.0002439:  log likelihood = 6357.63,  aic = -12709.25
```

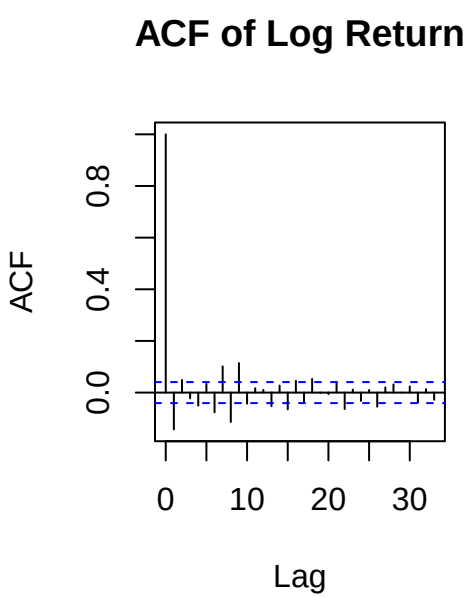
```r
ARMA11
```

```
Call:
arima(x = log_return, order = c(1, 0, 1))

Coefficients:
          ar1     ma1  intercept
      -0.3167  0.1774     -7e-04
s.e.   0.1124  0.1160      3e-04

sigma^2 estimated as 0.0002434:  log likelihood = 6360.17,  aic = -12712.34
```

```r
# 10. Compare AIC
AIC(AR1, MA1, ARMA11)
```

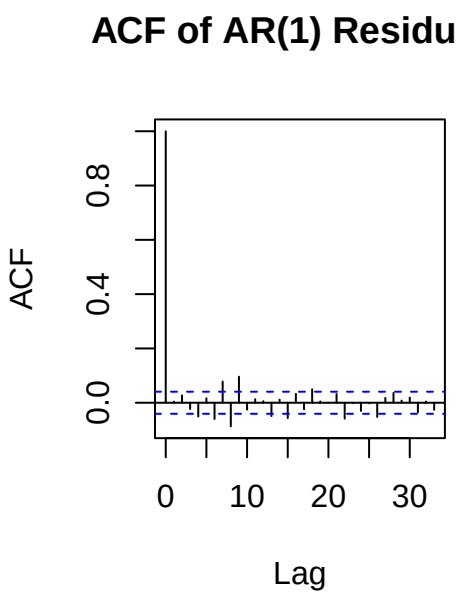
```
       df       AIC
AR1     3 -12712.45
MA1     3 -12709.25
ARMA11  4 -12712.34
```

```r
# 11. Check residuals of the best model manually
# (example here uses AR1; replace if another model has lower AIC)
par(mfrow = c(1, 2))
```




```r
acf(residuals(AR1), main = "ACF of AR(1) Residuals")

# 12. Ljung-Box test on residuals
Box.test(residuals(AR1), lag = 20, type = "Ljung-Box")
```

```
    Box-Ljung test

data:  residuals(AR1)
X-squared = 96.548, df = 20, p-value = 5.198e-12
```



Section 4

EDA

Table 1: Summary Statistics of Stock Price

Table 1: Summary Statistics of Stock Price

mean_price	sd_price	min_price	max_price
69.91683	33.95005	24.4	152.07

Table 1 shows the mean, standard deviation, minimum and maximum of the stock price between 2017/1/3 to 2026/3/27.

Transform Data

Figure 2 indicates that the time series is not stationary, as the ACFs are out of the boundary and decay very slowly. Thus, we need to transform the data to make it stationary. For stock data, taking the natural log is a common way to transform the data.

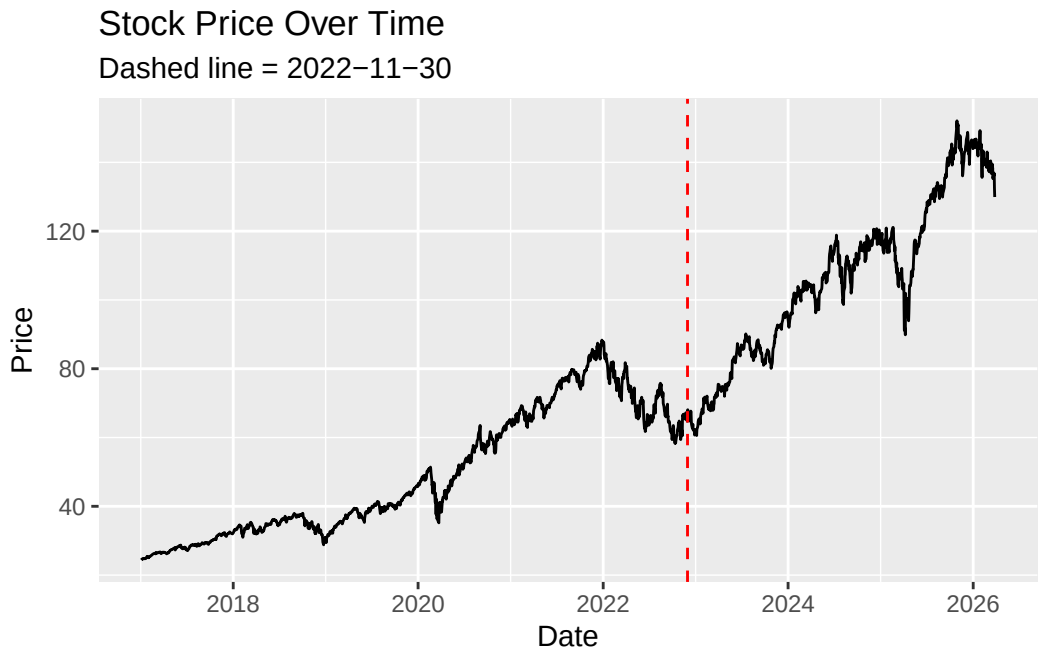


Figure 1

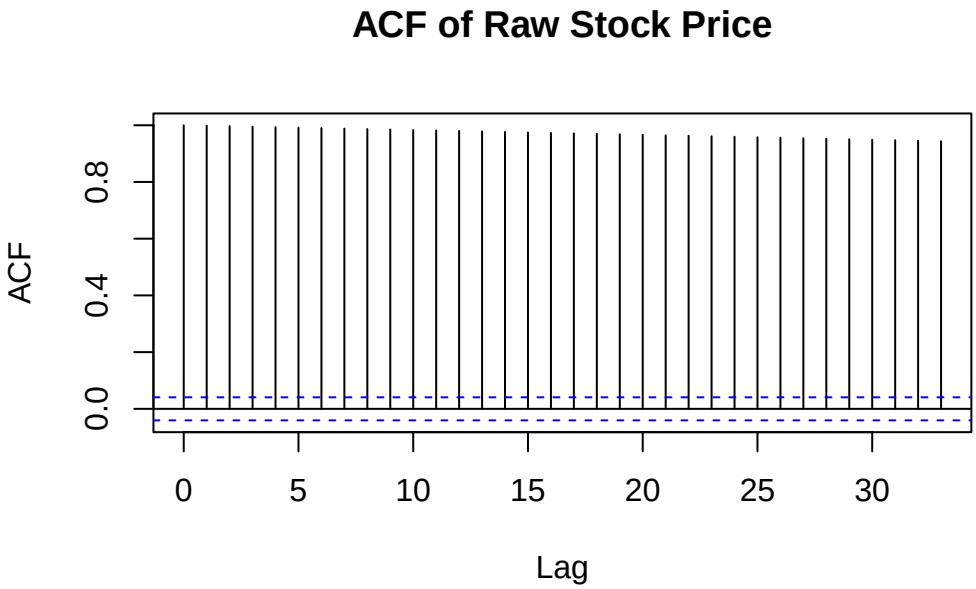


Figure 2

```r
# log return the raw data
#| eval: true
#| echo: false
#| message: false
#| warning: false
data <- data |>
  mutate(log_price = log(Price),
         log_return = c(NA, diff(log_price)))

before_data <- data |> filter(Date < cutoff)
after_data  <- data |> filter(Date >= cutoff)
```

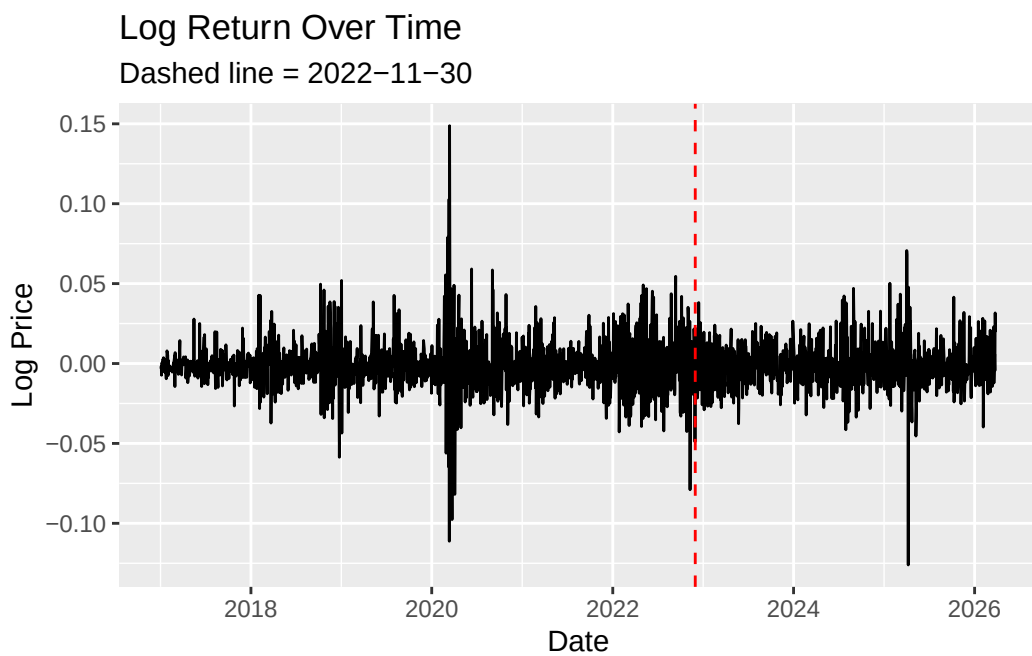


Figure 3

Figure Figure 3 shows the log return over time. The data fluctuates around zero, which suggests the mean is around zero. Figure Figure 4 shows the ACF of log return. Most ACFs fall within the bound. Thus, the log return time series looks stationary.

Compare log return

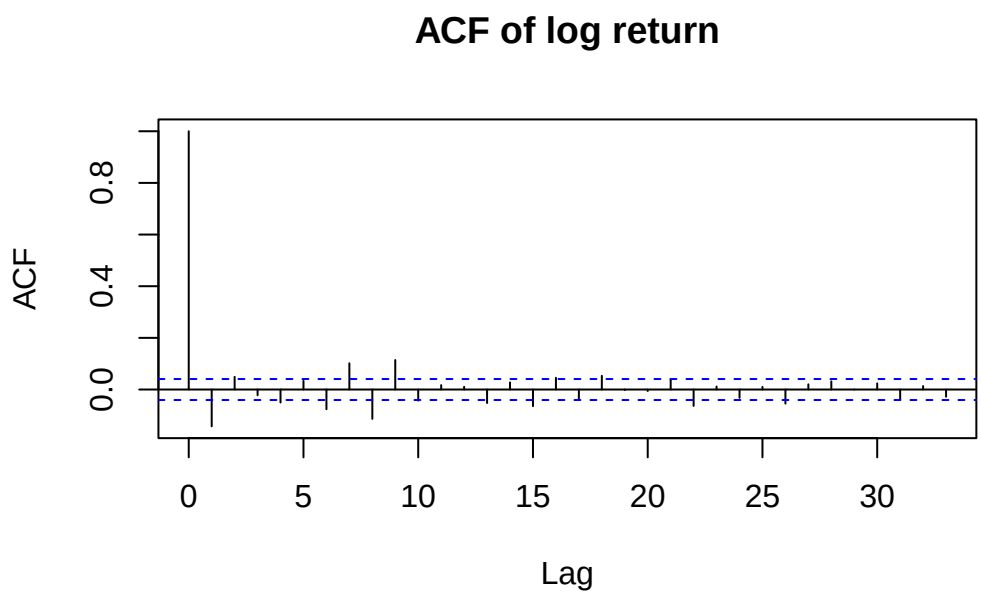


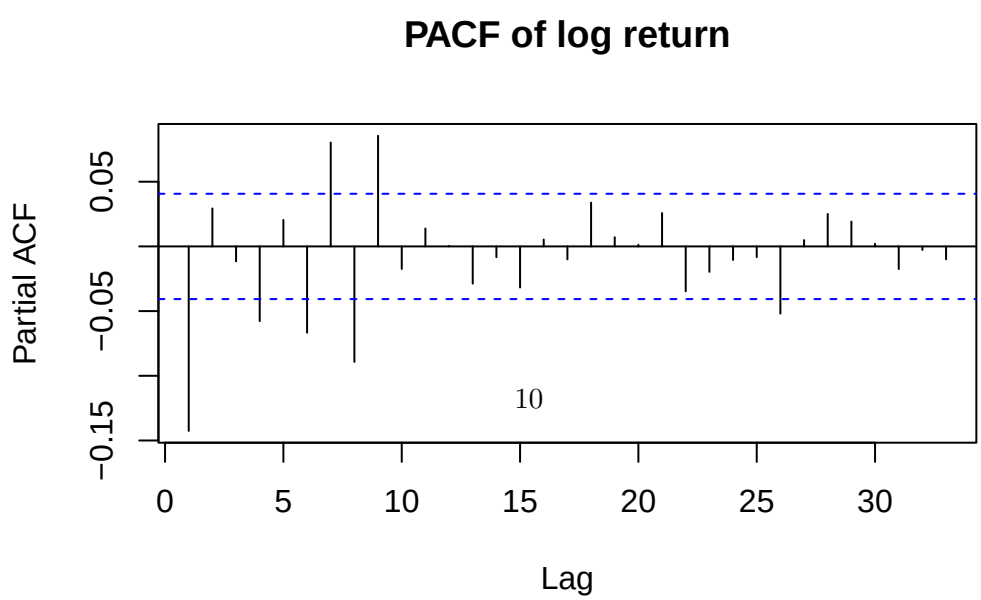
Figure 4

Table 2: Summary Statistics of Stock Price

period	Mean_Return	SD_Return
After	-0.00078	0.01456
Before	-0.00069	0.01641

According to table Table 2, the mean log return before and after the appearance of ChatGPT is both close to zero.

Fit Model



```
Call:
arima(x = data$log_return, order = c(1, 0, 0))

Coefficients:
          ar1  intercept
      -0.1426     -7e-04
s.e.   0.0206      3e-04

sigma^2 estimated as 0.0002436:  log likelihood = 6359.22,  aic = -12712.45
```

```r
#MA(1)
MA1 = arima(data$log_return, order = c(0,0,1))
print(MA1)
```

```
Call:
arima(x = data$log_return, order = c(0, 0, 1))

Coefficients:
          ma1  intercept
      -0.1337     -7e-04
s.e.   0.0199      3e-04

sigma^2 estimated as 0.0002439:  log likelihood = 6357.63,  aic = -12709.25
```

```r
#ARMA(1,1)
ARMA1 = arima(data$log_return, order = c(1,0,1))
print(ARMA1)
```

```
Call:
arima(x = data$log_return, order = c(1, 0, 1))

Coefficients:
          ar1     ma1  intercept
      -0.3122  0.1725     -7e-04
s.e.   0.1136  0.1171      3e-04

sigma^2 estimated as 0.0002434:  log likelihood = 6360.17,  aic = -12712.34
```

Forcasting

We are interested in how significant the data after 2022/11/30, when AI stared its prevalence, is to forecasting future stock market prices. We will being using data frames of different time sets: 2017/1/3 - 2026/3/27, 2017/1/3 - 2022/11/30 and 2022/11/30 - 2026/3/27. The forecast models we will be using are five "benchmark" methods: Mean method, Naive method and Seasonal naive method. Drift method and decomposition method seem unnecessary with log return data because we already removed the trend. (Note: We can do a regression forecast if we have time)

First, we aimed to examine whether AI has had an unexpected impact on the stock market and contributed to its fluctuations. We would like to use data from 2017/1/3 - 2022/11/30 to forecast data after 2022/11/30.

Table 3: Forecast Errors

Table 3: Forecast Errors

Method	MAE	RMSE
Mean	0.01049	0.01457
Naive	0.01241	0.01597
Seasonal	0.02161	0.02880

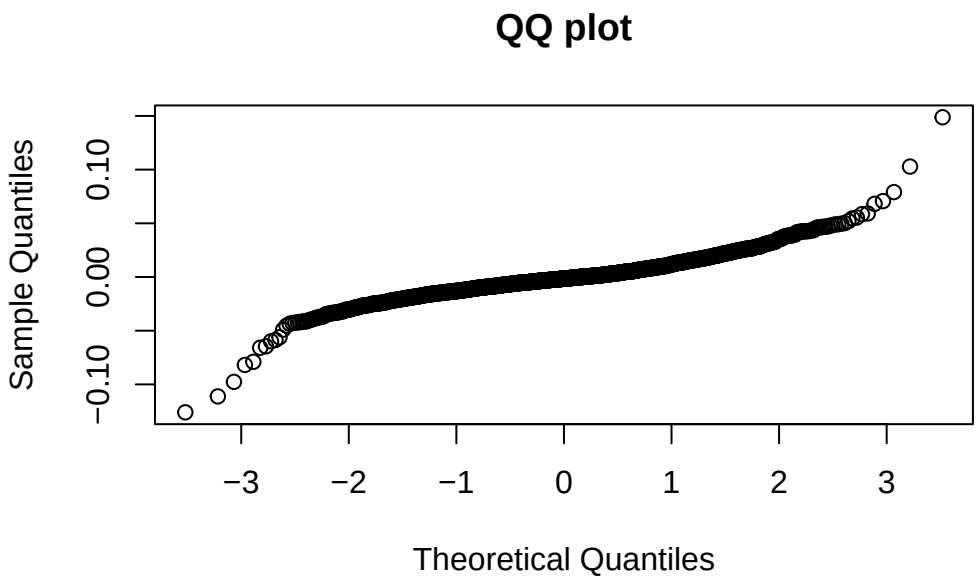
According to Table 3, all the scaled dependent errors are small, even for these easier approaches. It appears that the data prior to 2022/11/30 can effectively predict the data following the cutoff point. Furthermore, this suggests that AI have not yet contribute a significant impact in the stock market. As a result, we will use the entire time frame, allocating 80% for training and 20% for testing, to build a forecast model in predict prices in 2026.

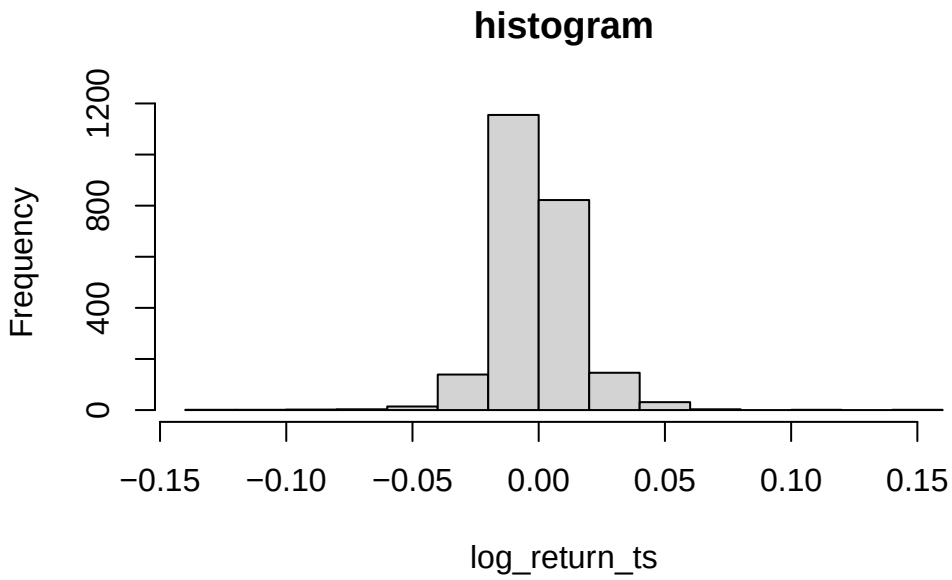
Table 4: Forecast Errors-full model

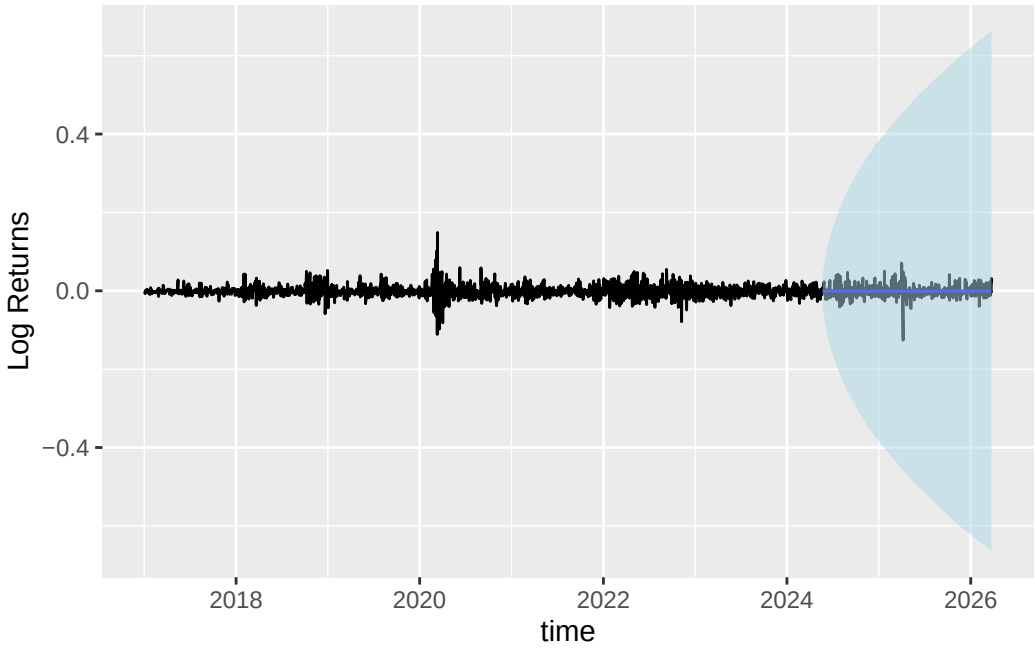
Table 4: Forecast Errors-full model

Method	MAE	RMSE
Mean	0.01128	0.01631
Naive	0.01459	0.01845
Seasonal	0.01311	0.01754

Table 4 shows the mean method is the best with lowest MAE and RMSE. We did not put MAPE here for comparison, because our log return data moves around 0, where some small values here make MAPE enormous and not as a good parameter to look at.





Next, we would like to also make a prediction interval for this forecast model. Our ACF for log return looks acceptable as previously showed. The histogram looks normal, however qq-plot reveal that log return does not satisfy the linearity assumption. This suggests that our prediction interval is off.

(Note: actually I just realize there is probably no need to do prediction on this, because it is around 0; we are wondering if we can forecast in ARMA)